

Los comandos y trucos esenciales para el día a día — desde tu primer commit hasta colaborar con Pull Requests en GitHub.

EN ESTA HOJA

01 • Git

02 • GitHub

03 • Markdown

EMPIEZA EN 4 PASOS

1 git init

Crea el repositorio en tu carpeta.



2 git add .

Prepara los cambios (stage).



3 git commit -m

Guarda una versión con mensaje.



4 git push

Súbelo a GitHub.

PARTE 1 Git — el control de versiones



CONFIGURACIÓN INICIAL · una sola vez

```
$ git config --global user.name "Tu Nombre"
```

Nombre que aparecerá en tus commits.

```
$ git config --global user.email "tu@correo.com"
```

Correo asociado a tus commits.

```
$ git config --global init.defaultBranch main
```

Usa "main" como rama por defecto.

```
$ git config --list
```

Muestra toda tu configuración actual.



HISTORIAL Y DIFERENCIAS

```
$ git log --oneline
```

Historial compacto: una línea por commit.

```
$ git log --oneline --graph --all
```

Historial visual con todas las ramas.

```
$ git diff
```

Cambios que aún NO están en el stage.

```
$ git diff --staged
```

Cambios que ya están en el stage.



CREAR Y OBTENER REPOSITORIOS

```
$ git init
```

Inicializa un repositorio en la carpeta actual.

```
$ git clone <url>
```

Descarga un repo remoto con todo su historial.

```
$ git clone --depth 1 <url>
```

Clon superficial: solo el último commit, mucho más rápido para repos grandes.

```
$ git clone <url> <carpeta>
```

Clona dentro de una carpeta con el nombre que tú elijas.



DESHACER COSAS · los salvavidas

```
$ git restore <archivo>
```

Descarta cambios del archivo (vuelve al último commit).

```
$ git restore --staged <archivo>
```

Saca el archivo del stage sin perder los cambios.

```
$ git restore .
```

Descarta TODOS los cambios sin guardar y vuelve al último commit — útil para desechar todo de golpe.

```
$ git commit --amend -m "nuevo mensaje"
```

Corrige el mensaje del último commit.



EL DÍA A DÍA: CAMBIOS Y COMMITS

```
$ git status
```

Estado de los archivos (modificados, en stage, sin seguimiento).

```
$ git add <archivo>
```

Añade un archivo específico al stage.

```
$ git add .
```

Añade todos los cambios al stage.

```
$ git commit -m "mensaje"
```

Crea un commit con lo que está en el stage.

```
$ git commit -am "mensaje"
```

Add + commit en un paso (archivos ya rastreados).



RESET, REVERT Y REFLOG

```
$ git reset --soft HEAD~1
```

Deshace el último commit, conserva cambios en stage.

```
$ git reset --mixed HEAD~1
```

Deshace el commit, cambios quedan sin stage (default).

```
$ git reset --hard <commit>
```

⚠ **Cuidado:** regresa a un commit descartando TODO lo posterior.

```
$ git revert <commit>
```

Crea un commit que revierte otro (seguro para remotos).

```
$ git reflog
```

Historial de TODO: tu máquina del tiempo para recuperar commits "perdidos".

 RAMAS

```
$ git branch
```

Lista las ramas (el * indica dónde estás).

```
$ git switch -c <rama>
```

Crea una rama nueva y se mueve a ella.

```
$ git switch <rama>
```

Cambia a otra rama.

```
$ git merge <rama>
```

Une la rama indicada con la rama actual.

```
$ git branch -d <rama>
```

Elimina una rama ya fusionada.

```
$ git branch -m <nuevo-nombre>
```

Renombra la rama actual.

 RESOLVER CONFLICTOS DE MERGE

Cuando dos ramas cambian la misma línea, Git marca el conflicto así en el archivo:

```
<<<<<< HEAD
tu versión
=====
la versión entrante
>>>>>> otra-rama
```

1. Edita el archivo y deja el código final (borra los marcadores).
2. `git add <archivo>` marca el conflicto resuelto.
3. `git commit` cierra el merge.

```
$ git merge --abort
```

Cancela el merge y vuelve todo a como estaba.

 STASH · trabajo a medias

```
$ git stash
```

Guarda los cambios en curso y limpia el directorio.

```
$ git stash list
```

Muestra todos los stashes guardados.

```
$ git stash pop
```

Recupera el último stash y lo elimina de la lista.

```
$ git stash apply
```

Recupera el último stash pero lo conserva.

```
$ git stash drop
```

Elimina el último stash sin aplicarlo.

 REBASE · reescribir historia

```
$ git rebase main
```

Pone tus commits encima de main.

```
$ git rebase -i HEAD~3
```

Rebase interactivo: squash, reword, edit.

⚠ **Regla de oro:** nunca hagas rebase de commits que ya subiste a un remoto compartido.

 TAGS · etiquetas de versión

```
$ git tag
```

Lista todas las etiquetas.

```
$ git tag -a v1.0.0 -m "Versión 1.0.0"
```

Crea una etiqueta anotada (la recomendada).

```
$ git tag -d v1.0.0
```

Elimina una etiqueta local.

 REMOTOS · trabajar con GitHub

```
$ git remote add origin <url>
```

Conecta tu repo local con uno remoto.

```
$ git remote -v
```

Muestra los remotos configurados.

```
$ git push -u origin main
```

Sube la rama y la vincula (solo la primera vez).

```
$ git push
```

Sube tus commits al remoto.

```
$ git push --tags
```

Sube también las etiquetas.

 TRAER CAMBIOS DEL REMOTO

```
$ git pull
```

Descarga y fusiona los cambios del remoto.

```
$ git pull --rebase
```

Descarga y pone tus commits encima (historial limpio).

```
$ git fetch
```

Descarga los cambios del remoto SIN fusionarlos.

🗑 EL ARCHIVO .GITIGNORE

```
node_modules/
```

Ignora una carpeta completa.

```
*.log
```

Todos los archivos con esa extensión.

```
dist/
```

Carpetas de build.

```
.env
```

Variables de entorno — ¡nunca subas secretos!

🔍 INSPECCIONAR EL HISTORIAL

```
$ git blame <archivo>
```

Muestra quién y en qué commit cambió cada línea.

```
$ git log -p
```

Historial con el diff completo de cada commit.

```
$ git log --author="nombre"
```

Filtra los commits por autor.

```
$ git log -- <archivo>
```

Solo los commits que tocaron ese archivo.

```
$ git shortlog -sn
```

Ranking de commits por persona (útil para créditos).

```
$ git bisect start
```

Búsqueda binaria para hallar el commit que introdujo un bug.

🔎 BUSCAR EN EL HISTORIAL

```
$ git log --grep="texto"
```

Busca commits cuyo mensaje contenga ese texto.

```
$ git log -S "función"
```

Encuentra cuándo se agregó o quitó un fragmento de código.

```
$ git diff rama1..rama2
```

Compara dos ramas entre sí.

```
$ git show <commit>:archivo
```

Ver un archivo tal como estaba en ese commit.

📁 MOVER, BORRAR Y LIMPIAR

```
$ git mv <viejo> <nuevo>
```

Renombra o mueve un archivo conservando su historial.

```
$ git rm <archivo>
```

Borra el archivo y lo saca del seguimiento.

```
$ git rm --cached <archivo>
```

Deja de rastrear un archivo sin borrarlo del disco (ideal tras editar .gitignore).

```
$ git clean -fd
```

⚠ **Cuidado:** elimina archivos y carpetas sin seguimiento de forma permanente.

📁 WORKTREES · varias ramas a la vez

```
$ git worktree add ../hotfix rama
```

Abre esa rama en otra carpeta, sin tocar tu trabajo actual.

```
$ git worktree add -b nueva ../dir
```

Crea una rama nueva directamente en su propia carpeta.

```
$ git worktree list
```

Muestra todas las carpetas de trabajo activas.

```
$ git worktree remove ../hotfix
```

Elimina el worktree cuando terminas (la rama se conserva).

Alternativa a git stash: revisa un PR o arregla un bug sin cambiar de rama.

> ALIAS · escribe menos

```
$ git config --global alias.st status
```

Ahora git st equivale a git status.

```
$ git config --global alias.lg "log --oneline --graph --all"
```

git lg: historial gráfico de todas las ramas en una línea.

```
$ git config --global --list
```

Revisa todos tus alias y ajustes globales.

Los alias más comunes: co → checkout, br → branch, ci → commit.

PARTE 2 GitHub — colaboración

🔗 FLUJO DE TRABAJO · Feature Branch

1. git switch -c mi-feature crea tu rama.
2. Trabaja y haz commits.
3. git push -u origin mi-feature sube la rama.
4. Abre un **Pull Request** en GitHub.
5. Revisión → correcciones → aprobación.
6. **Merge** del PR y elimina la rama.

🔗 COLABORAR EN PROYECTOS AJENOS · Fork

1. **Fork** copia el repo a tu cuenta.
2. git clone <url-de-tu-fork> clónalo.
3. Crea rama, haz cambios, push a TU fork.
4. Abre un **Pull Request** al repo original.
5. Mantén tu fork al día con **Sync fork**.



SINCRONIZAR UN FORK · upstream

```
$ git remote add upstream <url-original>
```

Conecta tu fork con el repositorio original.

```
$ git fetch upstream
```

Descarga los cambios nuevos del original.

```
$ git merge upstream/main
```

Intégralos en tu rama local.

```
$ git push origin main
```

Sube tu fork ya actualizado a GitHub.



TRUCOS CON ISSUES Y COMMITS

```
fixes #12 · closes #12
```

Cierra el issue #12 al hacer merge.

```
#12
```

Enlaza el commit con el issue (sin cerrarlo).

```
@usuario
```

Menciona y notifica a alguien.



GITHUB CLI · gh

```
$ gh auth login
```

Inicia sesión en GitHub desde la terminal.

```
$ gh repo create
```

Crea un repo sin abrir el navegador.

```
$ gh repo clone usuario/repo
```

Clona un repositorio.



GITHUB CLI · PRs e issues

```
$ gh pr create
```

Abre un Pull Request desde la terminal.

```
$ gh pr list
```

Lista los PRs abiertos.

```
$ gh pr checkout 12
```

Descarga el PR #12 para probarlo localmente.

```
$ gh issue create
```

Crea un issue.



ATAJOS DE TECLADO · github.com

t Buscador rápido de archivos del repo.

. Abre el repo en VS Code web (github.dev).

l Salta a una línea específica de un archivo.

b Ver blame (quién cambió cada línea).

y Convierte la URL en enlace permanente al commit.

? Muestra todos los atajos disponibles.



URLS Y TRUCOS ÚTILES

```
/usuario/repo/compare
```

Compara ramas o commits visualmente.

```
añade .patch o .diff a la URL
```

Ver un commit en texto plano.

README.md en un repo con tu mismo usuario se vuelve la portada de tu perfil.

CODEOWNERS asigna revisores automáticos a los PRs.



COMMITTS PROFESIONALES · Conventional Commits

```
feat:
```

Una funcionalidad nueva.

```
fix:
```

Corrección de un bug.

```
docs:
```

Cambios en documentación.

```
refactor:
```

Cambio que no agrega función ni corrige bug.

```
test:
```

Añadir o corregir pruebas.

```
chore:
```

Mantenimiento (dependencias, config).

Ejemplo:

```
feat: agregar validación de correo en el formulario
```



GLOSARIO ESENCIAL

Commit • foto guardada de tus cambios en un momento dado.

Repositorio • el proyecto completo con todo su historial.

Rama (branch) • línea de trabajo paralela e independiente.

Remoto • copia del repo en un servidor (p. ej. GitHub).

Origin • nombre por defecto de tu remoto principal.

HEAD • apuntador a tu commit/rama actual.

Stage • sala de espera de lo que irá al próximo commit.

Pull Request • propuesta para fusionar tu rama, con revisión.

Fork • copia de un repo ajeno en tu propia cuenta.

BUENAS PRÁCTICAS

- Commits pequeños y frecuentes, con mensajes claros en imperativo.
- Una rama por funcionalidad o bug; nunca trabajes directo en `main`.
- Haz `git pull` antes de empezar para evitar conflictos.
- Revisa el `git diff` antes de cada commit.
- Cuida un buen README y un `.gitignore` desde el inicio.

PARTE 3 Markdown — el lenguaje de los README, Issues y PRs

Markdown es texto plano con símbolos sencillos que GitHub convierte en formato. Escribe a la izquierda, así se ve a la derecha.

H TÍTULOS

```
# Título 1
```

De # a #####: seis niveles de encabezado, mayor a menor.

B ÉNFASIS EN EL TEXTO

```
**negrita**
```

Texto en **negrita**.

```
*cursiva*
```

Texto en *cursiva*.

```
~tachado~
```

Texto ~~tachado~~.

```
`código`
```

Código en `línea dentro de una frase`.

LISTAS

```
- item • * item
```

Lista con viñetas. Indenta dos espacios para anidar.

```
1. item
```

Lista numerada (GitHub renumera solo).

```
- [ ] tarea • - [x] hecha
```

Casillas de tarea; se marcan con un clic en Issues y PRs.

ENLACES E IMÁGENES

```
[texto](https://url)
```

Un enlace con el texto que quieras.

```
![alt](imagen.png)
```

Inserta una imagen. Arrástrala al editor y GitHub la sube por ti.

CITAS, LÍNEAS Y EMOJIS

```
> texto citado
```

Cita en bloque, con barra lateral.

```
---
```

Línea horizontal separadora.

```
:rocket: @usuario #12
```

Emoji por código, mención a persona y enlace a un issue/PR.

<> BLOQUES DE CÓDIGO

Tres comillas invertidas y el lenguaje activan el resaltado de sintaxis:

```
```js
const suma = (a, b) => a + b;
console.log(suma(2, 3));
```
```

TABLES

| | |
|---------|----------|
| Comando | Qué hace |
| add | agrega |
| commit | guarda |

La fila de guiones separa el encabezado. Usa :---, :--: o ---: para alinear.

⚠ AVISOS • alerts de GitHub

```
> [!NOTE]
> Información útil para el lector.

> [!WARNING]
> Algo que requiere atención.
```

También existen [!TIP], [!IMPORTANT] y [!CAUTION]; GitHub los pinta con color e icono.

💡 CONSEJOS DE MARKDOWN

- Deja una línea en blanco entre párrafos; si no, se unen en uno solo.
- Termina una línea con dos espacios para forzar un salto sin párrafo nuevo.
- Escapa un símbolo especial con \\ delante (p. ej. *).
- Previsualiza con la pestaña **Preview** antes de guardar.

🎓 APRENDE MÁS • devtalles

¿Quieres dominar Git & GitHub de verdad y seguir con desarrollo web, backend y más? Encuentra cursos completos en español en cursos.devtalles.com.